Getting Started with Databricks Community Edition

Theory

Databricks is a **cloud-based platform** for big data analytics and machine learning, built on **Apache Spark**.

In Data Engineering, the first steps usually involve:

1. **Setting up an environment** (Databricks workspace + cluster).
2. **Uploading data** (CSV/Parquet/JSON files).
3. **Accessing the file system** (DBFS – Databricks File System).

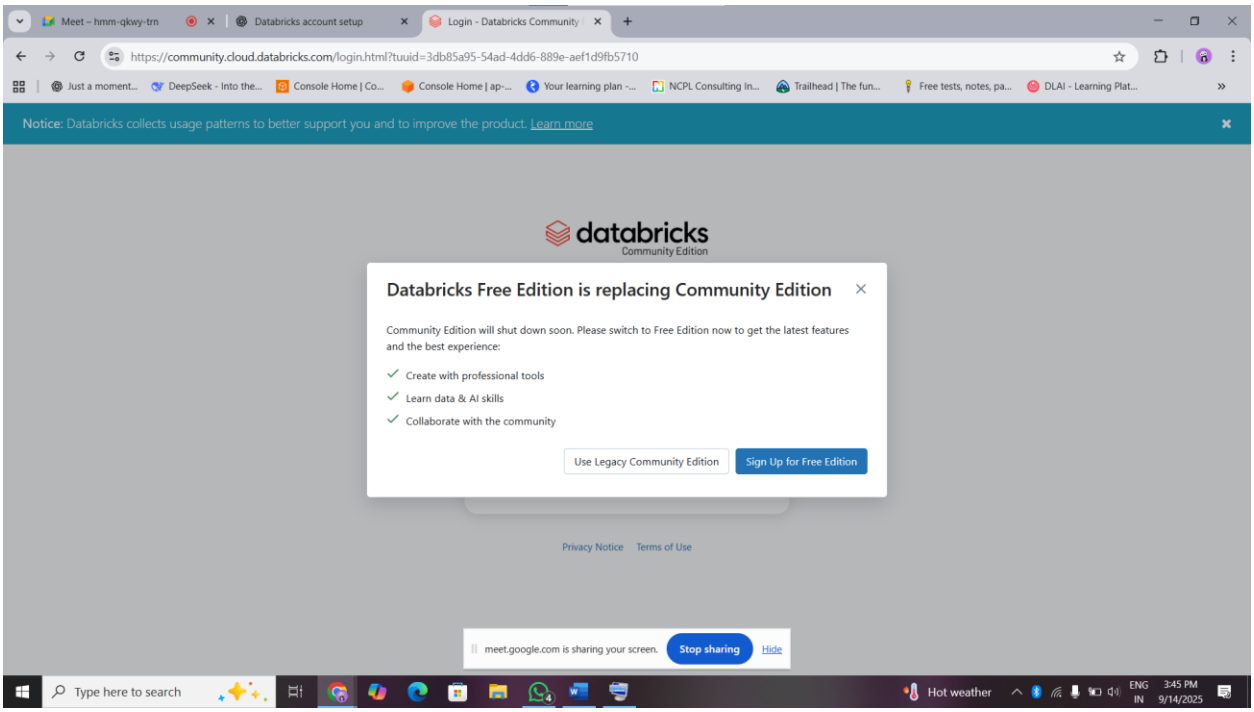
Key concept – DBFS (Databricks File System):

- A distributed file system built on cloud storage (S3 for AWS, ADLS for Azure, etc.).
- In Databricks CE, you interact with DBFS using dbutils.fs commands.
- /FileStore/tables/ is a default user-friendly location for uploaded files.

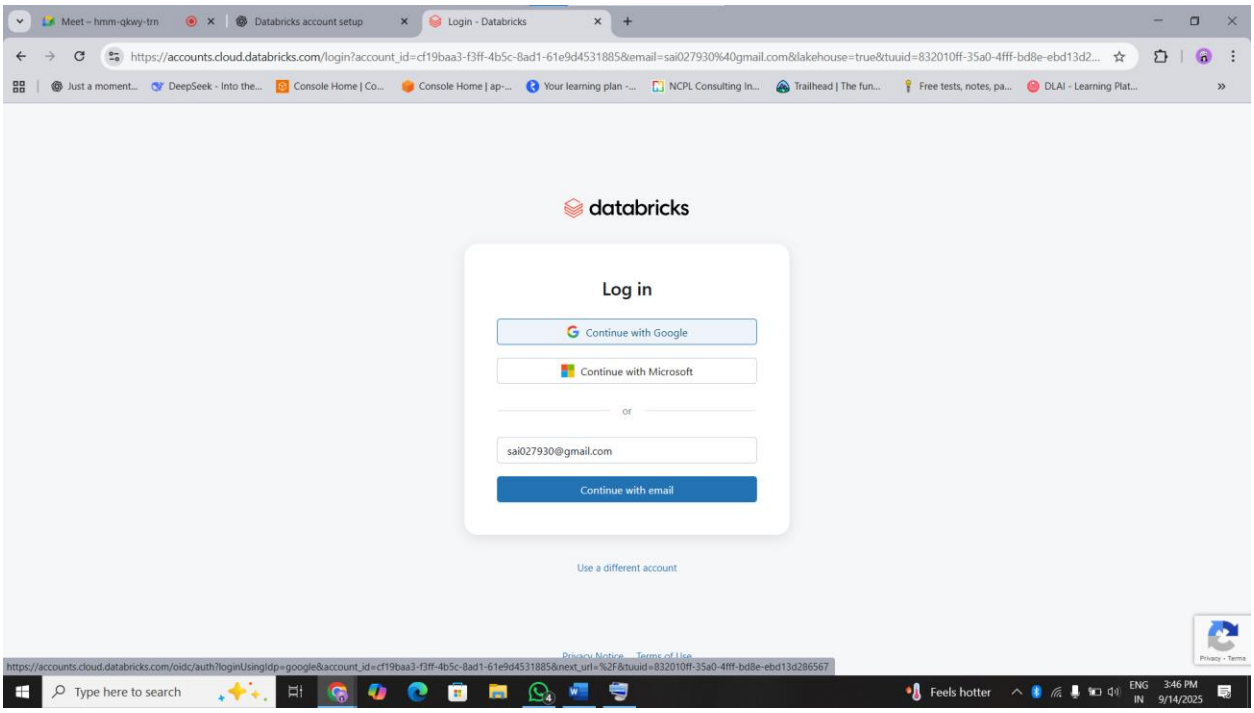
Steps

1. Create a Free Databricks Community Edition Account

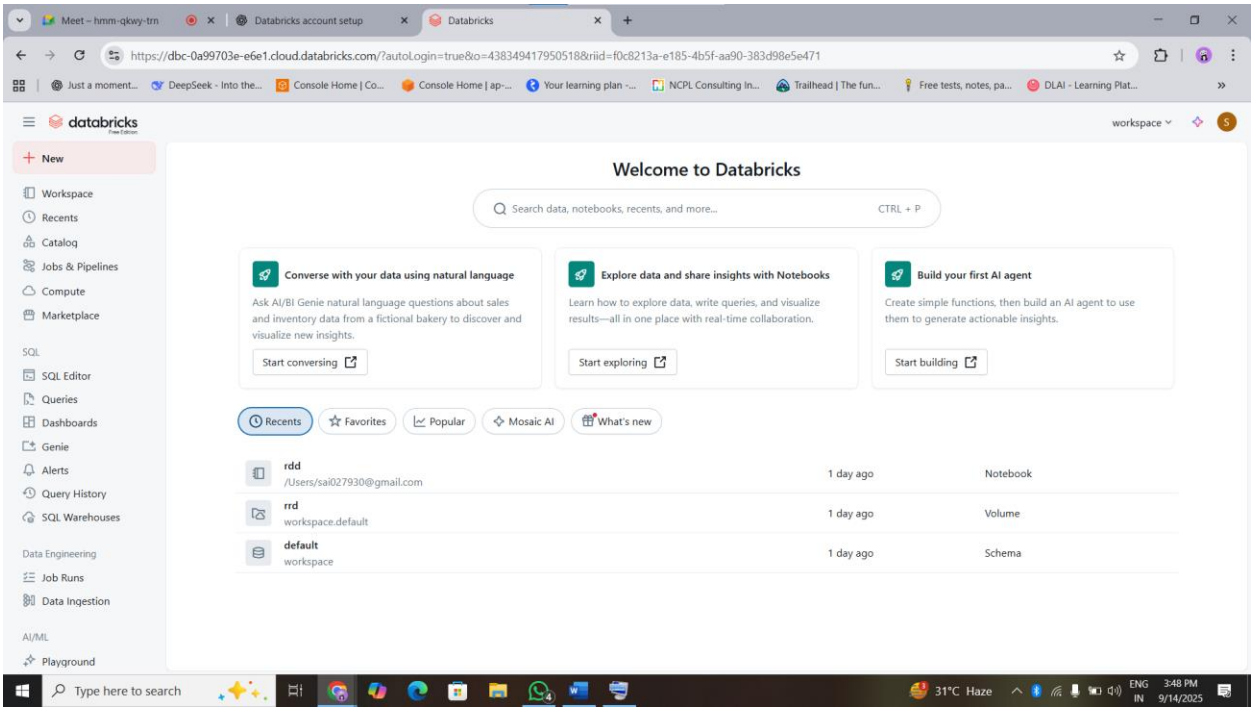
- Go to community.cloud.databricks.com.



- Click **Sign Up** → use a personal email (Gmail/Outlook/etc.).

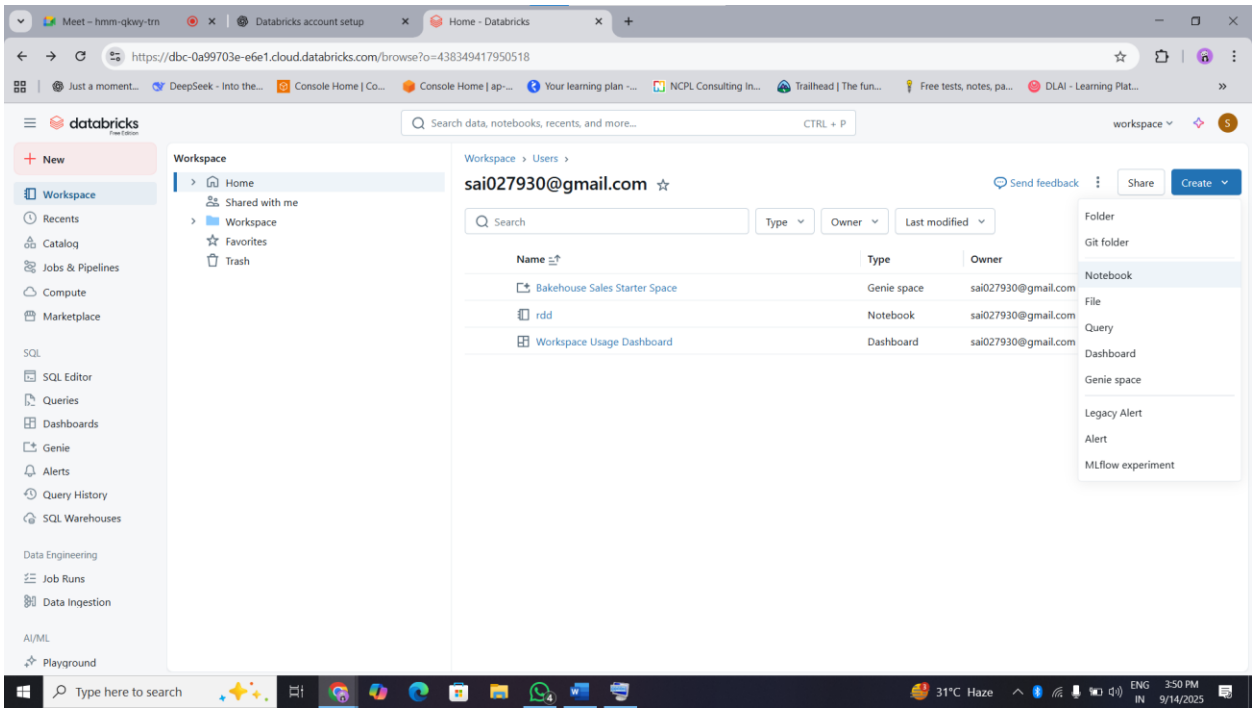


- Confirm email → log in to the **Databricks workspace**.



2. Navigate Workspace & Create a Notebook

- In the sidebar → **Workspace → Users → your_username**.
- Click **Create → Notebook**.

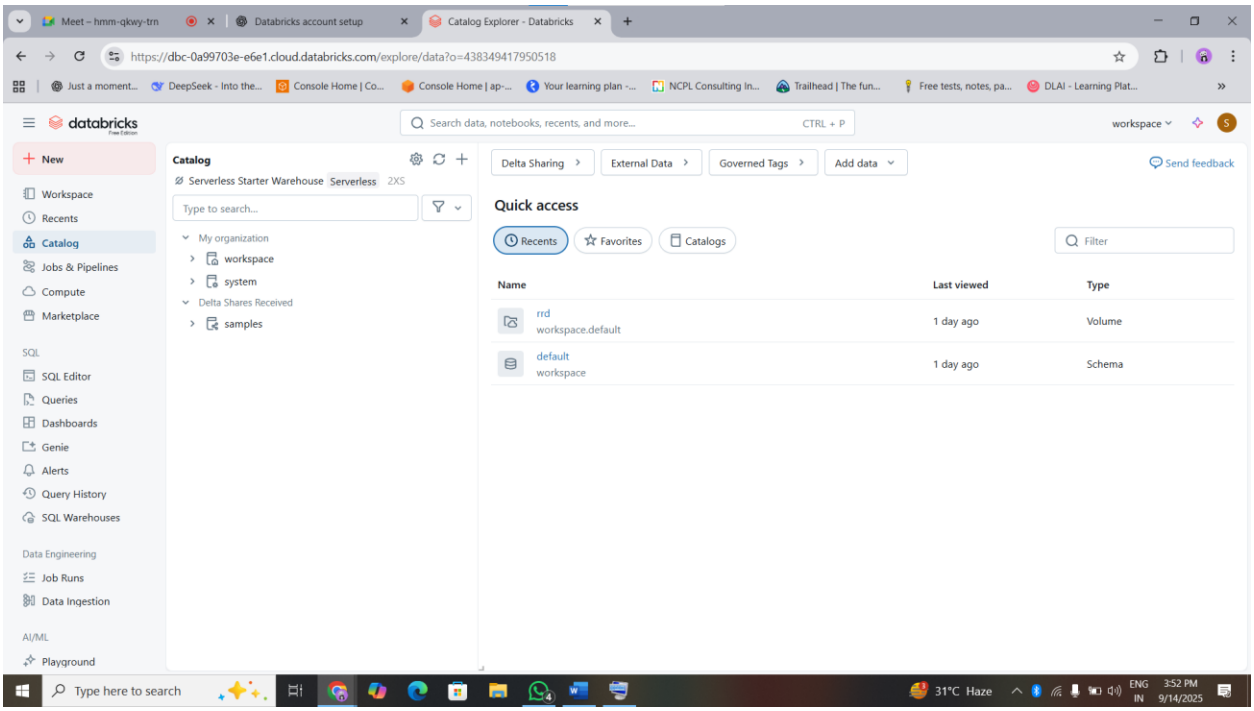


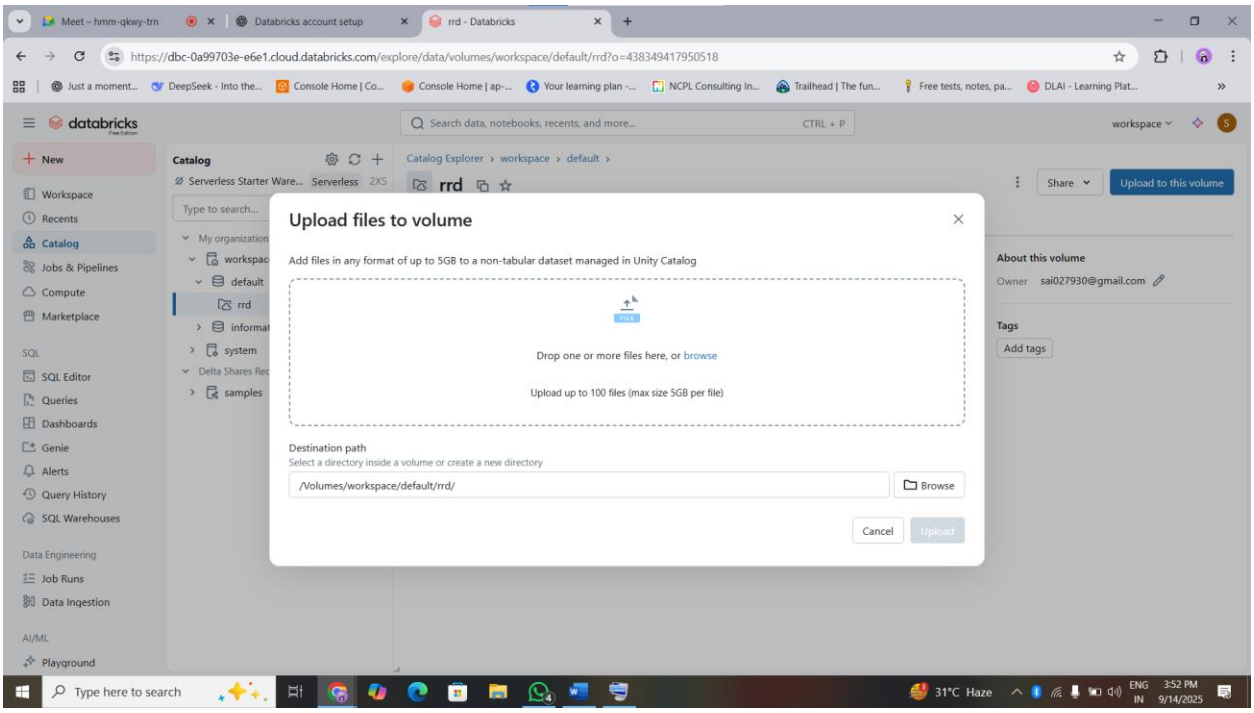
- Name the notebook (e.g., rdd).
- Set **Default Language: Python**.
- Attach notebook to the default cluster (start it if stopped).

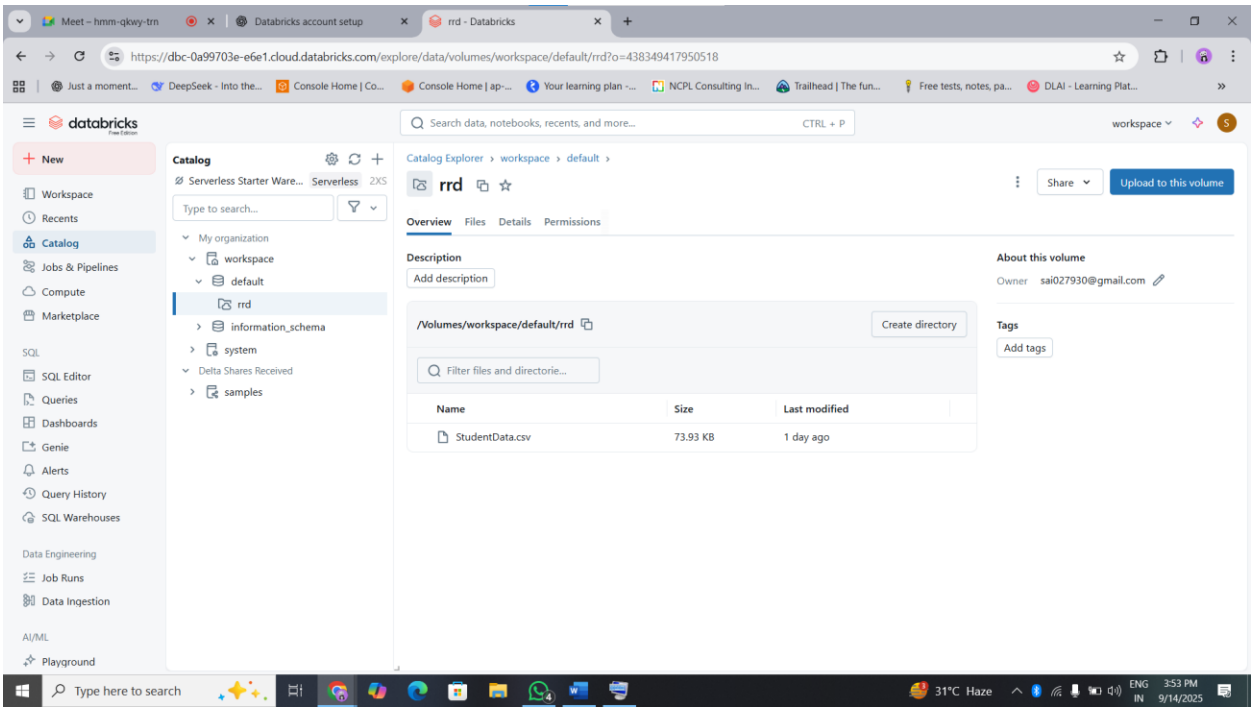
---

3. Upload CSV File to DBFS

- In the left sidebar → **Data → DBFS** (or click **Upload File**).
- Upload your student_data.csv.
- The file will appear under /FileStore/tables/.
    - Example path: /FileStore/tables/student_data.csv.


Meet – hmm-qkwy-trn
Databricks account setup
Catalog Explorer - Databricks
https://dbc-0a99703e-e6e1.cloud.databricks.com/explore/data?o=438349417950518
databricks
Search data, notebooks, recents, and more...
CTRL + P
workspace
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie
Alerts
Query History
SQL Warehouses
Data Engineering
Job Runs
Data Ingestion
AI/ML
Playground
Catalog
Serverless Starter Warehouse Serverless 2XS
Type to search...
My organization
workspace
system
Delta Shares Received
samples
Delta Sharing
External Data
Governed Tags
Add data
Send feedback
Quick access
Recents
Favorites
Catalogs
Filter
Name
Last viewed
Type
rrd
workspace.default
1 day ago
Volume
default
workspace
1 day ago
Schema
Type here to search
31°C Haze
ENG IN
3:52 PM 9/14/2025


Meet – hmm-qkwy-trn
Databricks account setup
rrd - Databricks
https://dbc-0a99703e-e6e1.cloud.databricks.com/explore/data/volumes/workspace/default/rrd?o=438349417950518
databricks
Search data, notebooks, recents, and more...
CTRL + P
workspace
Catalog Explorer › workspace › default ›
rrd
Share
Upload to this volume
Upload files to volume
Add files in any format of up to 5GB to a non-tabular dataset managed in Unity Catalog
Drop one or more files here, or browse
Upload up to 100 files (max size 5GB per file)
Destination path
Select a directory inside a volume or create a new directory
/Volumes/workspace/default/rrd/
Browse
Cancel
Upload
About this volume
Owner sai027930@gmail.com
Tags
Add tags
Type here to search
31°C Haze
ENG IN
3:52 PM 9/14/2025



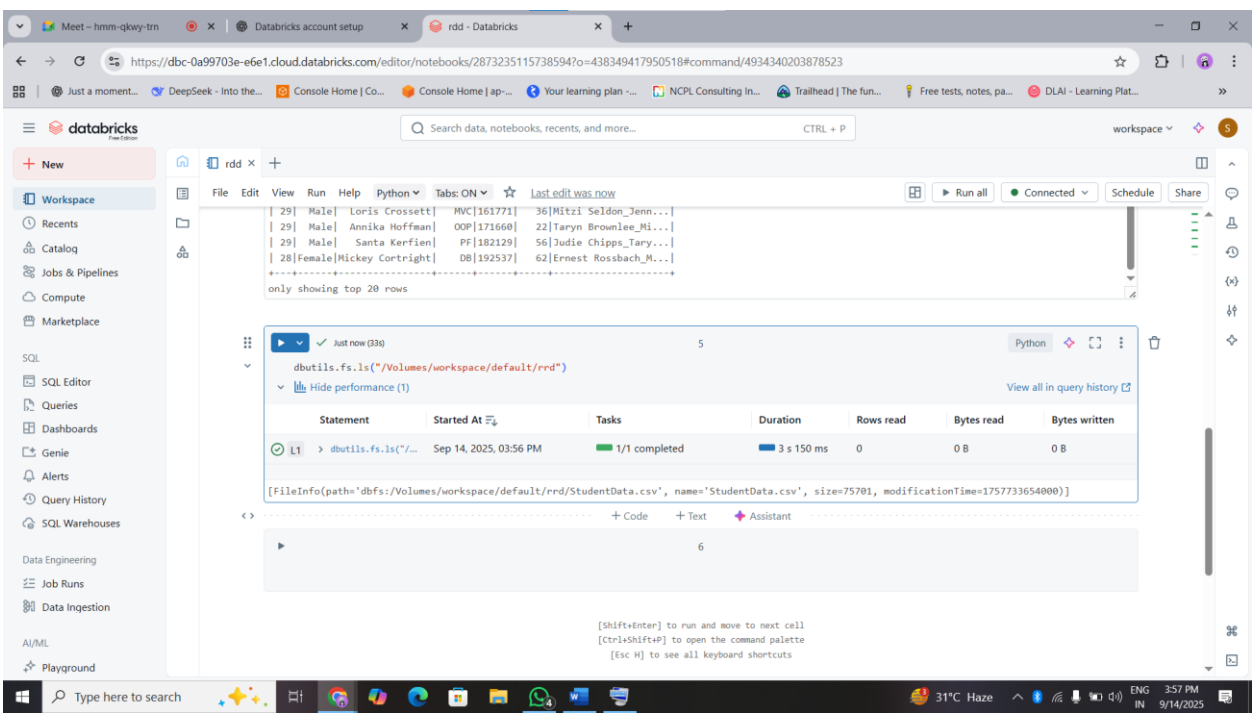
---

4. Verify File with dbutils

- Use dbutils.fs.ls() to list files inside DBFS and confirm your upload.

display(dbutils.fs.ls("/FileStore/tables/"))

This will show a table with file **name, path, size, and modification time**.



Data Engineering with Spark SQL & DataFrames

Theory

Apache Spark provides two powerful abstractions for Data Engineering:

1. **DataFrames**
   - Distributed collection of data organized into **rows** and **columns** (like a table in a database).
   - Built on top of RDDs but optimized with the Catalyst query optimizer.
   - Provides a high-level API for transformations (select, filter, groupBy, etc.).
2. **Spark SQL**
   - Enables querying structured data using **SQL syntax**.
   - DataFrames can be registered as temporary views → queried with SQL.
   - Spark SQL and DataFrame API are interchangeable (both run on the same execution engine).

Why use them?

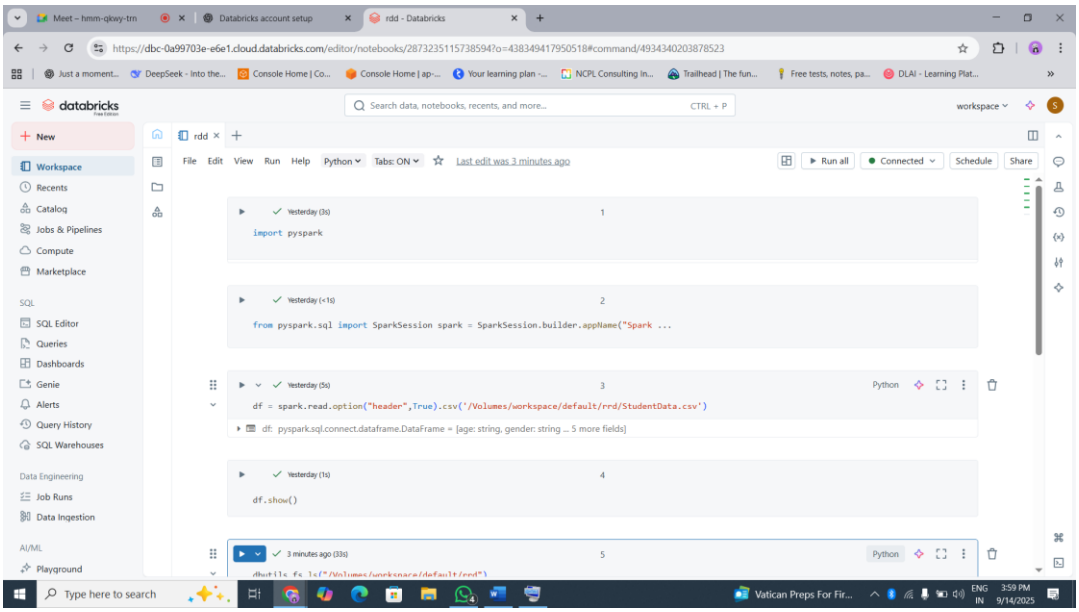
- Combine the familiarity of **SQL** with the power of **distributed computing**.
- Great for Data Engineering tasks: cleaning, transforming, aggregating, and preparing data for analysis.

Steps

1. Load CSV into a DataFrame

- Use spark.read.csv() to load raw data.
- Options like header=True (to use first row as column names) and inferSchema=True (to detect datatypes) help create a clean schema.

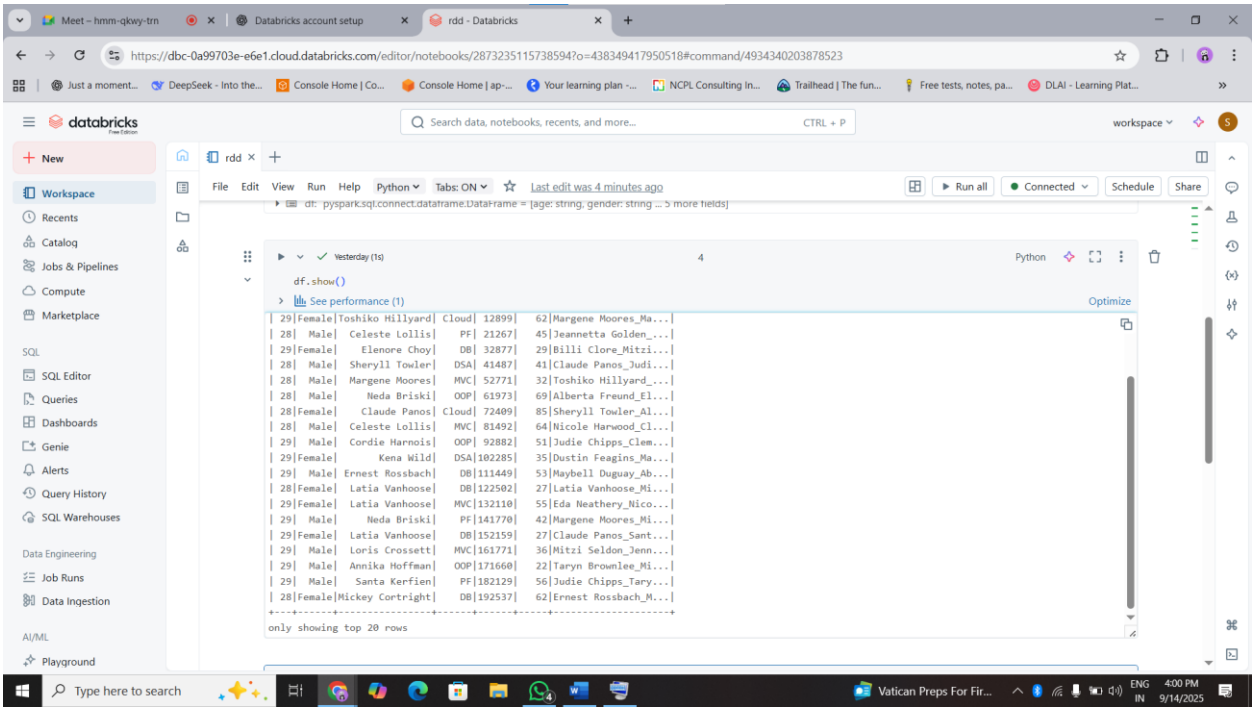
df = spark.read.csv("/FileStore/tables/student_data.csv", header=True, inferSchema=True)



2. Print Schema

- Understand structure (column names + datatypes).
- Helps validate if Spark correctly interpreted your data.

df.printSchema()



3. Perform Basic Transformations

- **Select** → Choose specific columns.
- **Filter** → Apply conditions to remove unwanted rows.
- **Add Column** → Derive new insights (e.g., pass/fail status).

from pyspark.sql.functions import col, when

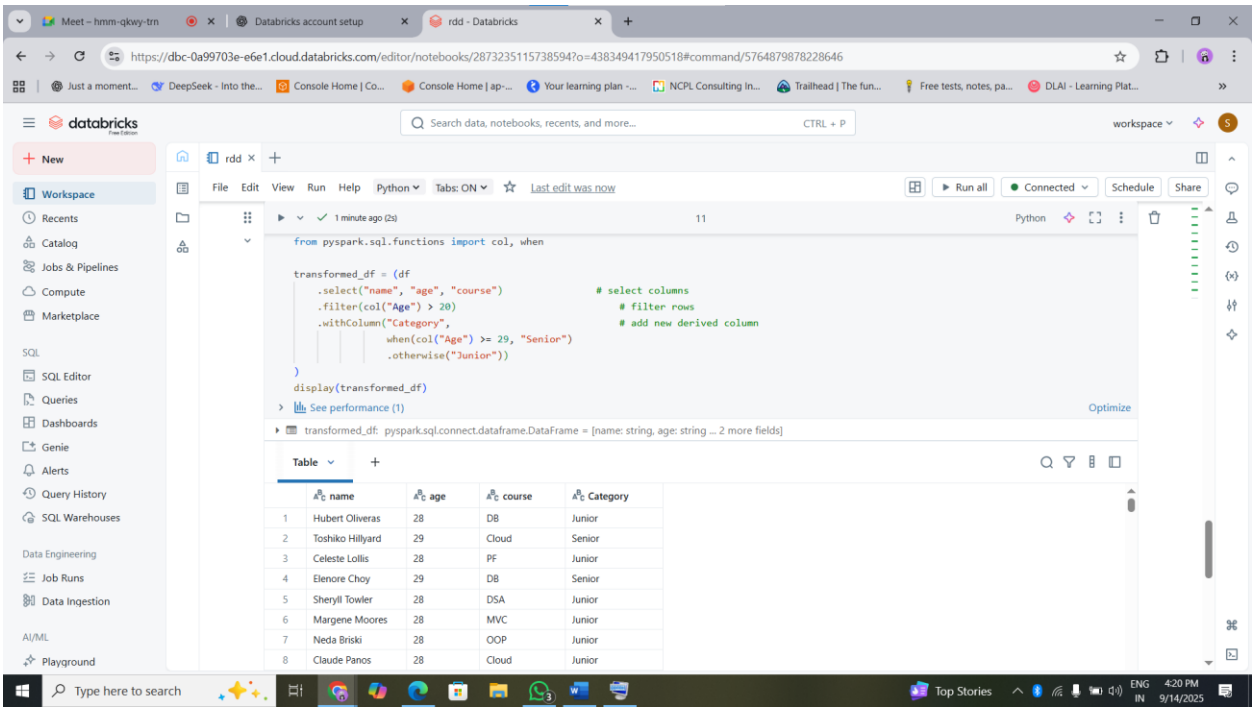
transformed_df = (df .select("name", "age", "course") # select columns .filter

(col("Age") > 20) # filter

rows .withColumn("Category", # add new derived column

when(col("Age") >= 29, "Senior") .otherwise("Junior")))

display(transformed_df)

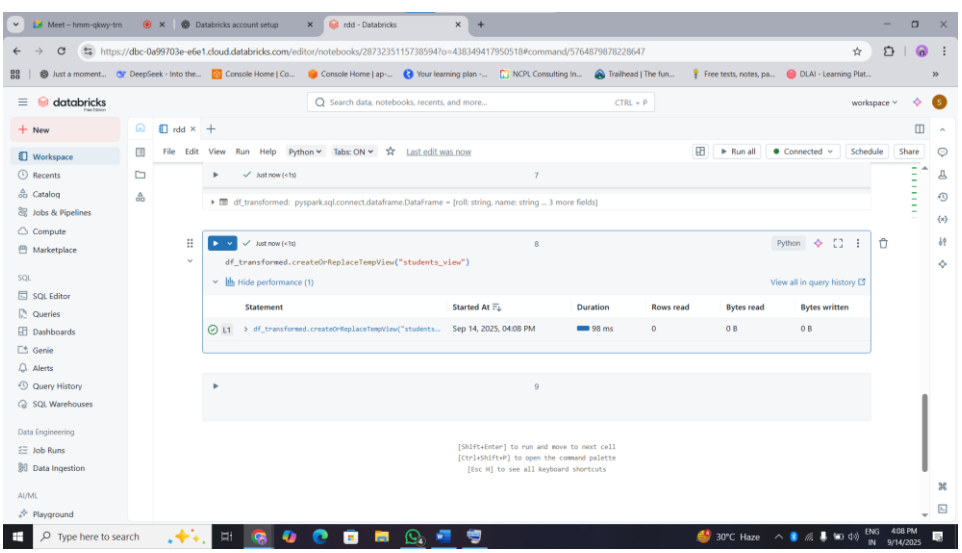


---

4. Register as Temporary View

- A DataFrame can be exposed as a **SQL table** inside Spark.
- Temporary view → lives only for the current session.

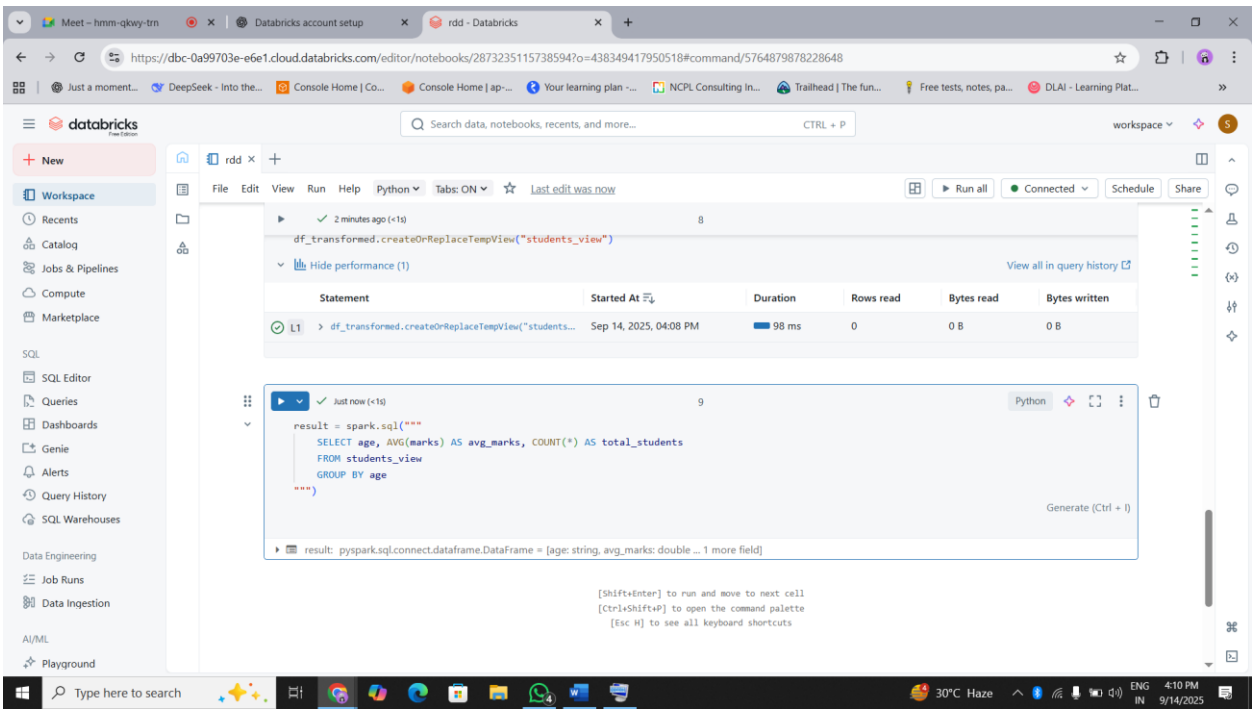
df_transformed.createOrReplaceTempView("students_view")



5. Run SQL Query

- Now we can query the DataFrame with SQL.
- Example: Grouping students by age and calculating average marks.

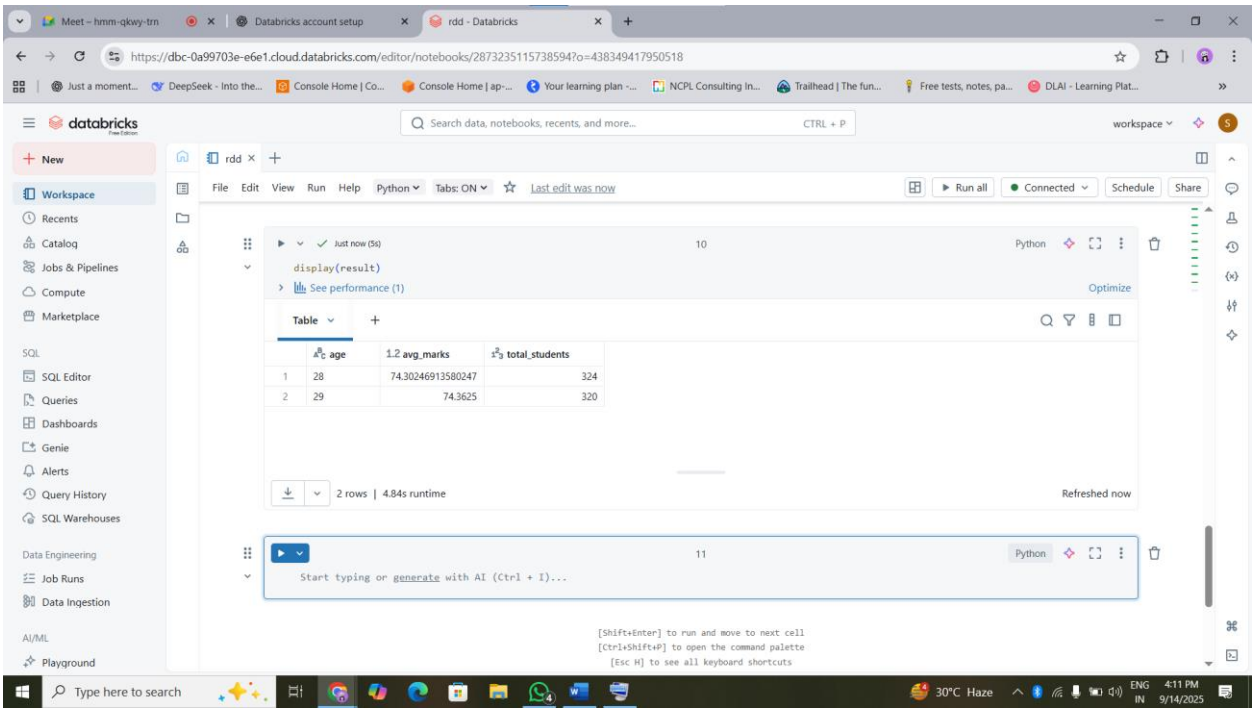
```
result = spark.sql("""
    SELECT age, AVG(marks) AS avg_marks, COUNT(*) AS total_students
    FROM students_view
    GROUP BY age
""")
```



6. Display Results

- In Databricks, display() gives a neat table or chart view.

display(result)



End Result:

- You loaded raw data → understood schema → cleaned & transformed → registered as view → analyzed with SQL → visualized results.

Understanding Delta Lake

Theory

Delta Lake is an open-source **storage layer** built on top of Parquet that adds **reliability, performance, and ACID transactions** to your data lake.

Core Features:

1. **ACID Transactions** → ensures reliable writes and reads.
2. **Time Travel** → query older versions of your data.
3. **Schema Enforcement & Evolution** → prevents corrupt data and allows schema changes in a controlled way.
4. **Unify Batch + Streaming** → same table works for both use cases.

Steps

1. Write DataFrame as a Delta Table

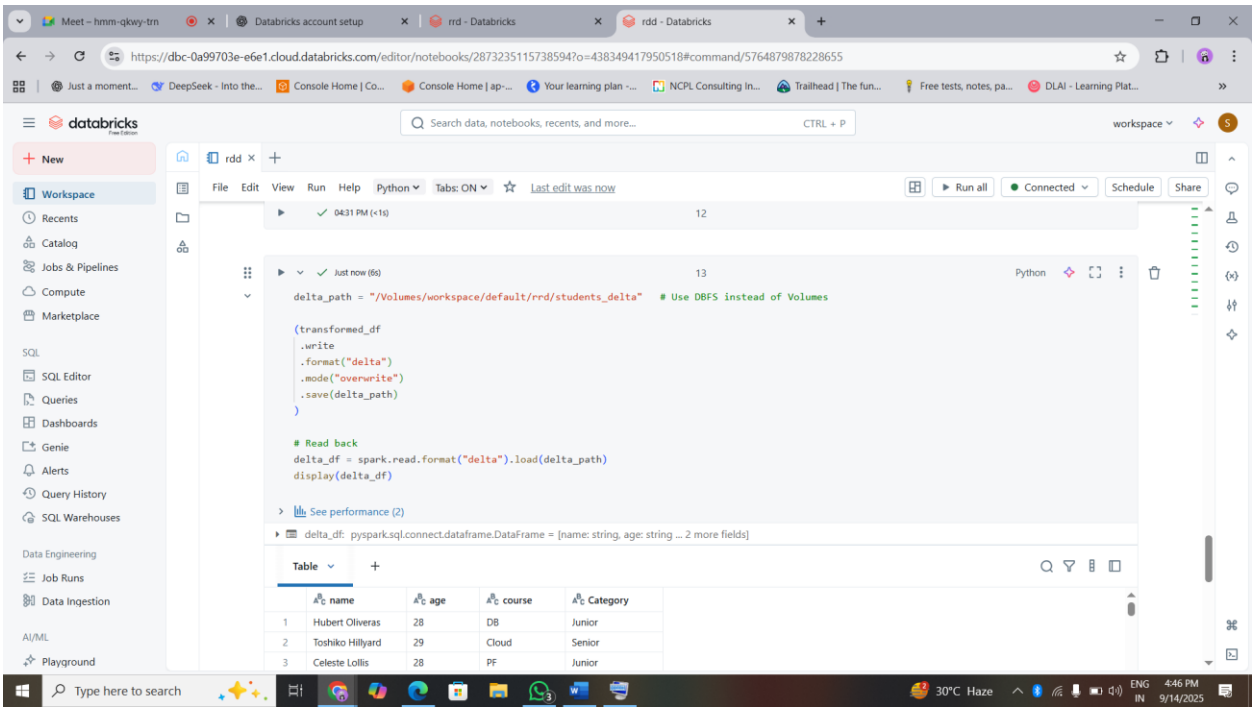
Take your transformed DataFrame from Task 2 (e.g., df_transformed) and save it in Delta format.

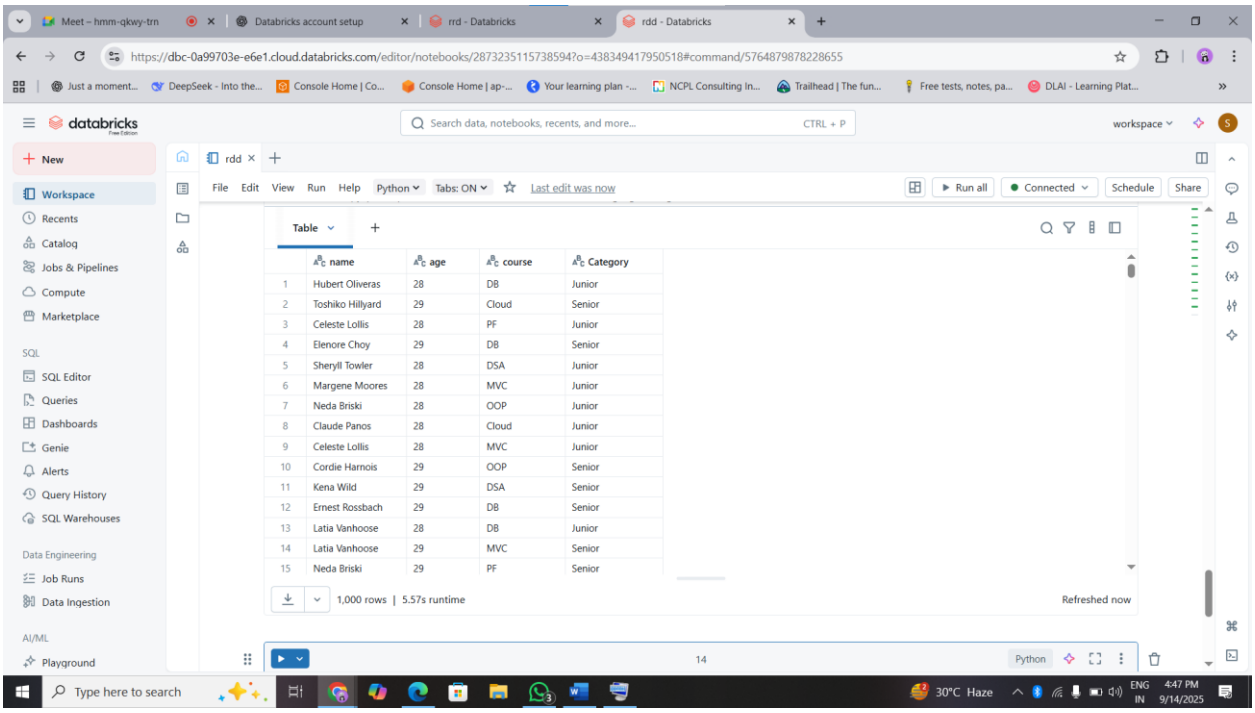
```
# Define Delta path in DBFS
delta_path = "/FileStore/tables/students_delta"

# Write DataFrame as Delta table
df_transformed.write.format("delta").mode("overwrite").save(delta_path)
```

Verify contents:

```
display(dbutils.fs.ls(delta_path))
```


rdd - Databricks
https://dbc-0a99703e-e6e1.cloud.databricks.com/editor/notebooks/2873235115738594?o=4383494179505 18#command/5764879878228655
File Edit View Run Help Python Tabs: ON Last edit was now
Run all Connected Schedule Share
Just now (6s) 13
delta_path = "/Volumes/workspace/default/rrd/students_delta" # Use DBFS instead of Volumes
(transformed_df
.write
.format("delta")
.mode("overwrite")
.save(delta_path)
)
# Read back
delta_df = spark.read.format("delta").load(delta_path)
display(delta_df)
See performance (2)
delta_df: pyspark.sql.connect.dataframe.DataFrame = [name: string, age: string ... 2 more fields]
Table
name age course Category
1 Hubert Oliveras 28 DB Junior
2 Toshiko Hillyard 29 Cloud Senior
3 Celeste Lollis 28 PF Junior


rdd - Databricks
https://dbc-0a99703e-e6e1.cloud.databricks.com/editor/notebooks/2873235115738594?o=4383494179505 18#command/5764879878228655
File Edit View Run Help Python Tabs: ON Last edit was now
Run all Connected Schedule Share
Table
name age course Category
1 Hubert Oliveras 28 DB Junior
2 Toshiko Hillyard 29 Cloud Senior
3 Celeste Lollis 28 PF Junior
4 Elenore Choy 29 DB Senior
5 Sheryll Towler 28 DSA Junior
6 Margene Moores 28 MVC Junior
7 Neda Briski 28 OOP Junior
8 Claude Panos 28 Cloud Junior
9 Celeste Lollis 28 MVC Junior
10 Cordie Harnois 29 OOP Senior
11 Kena Wild 29 DSA Senior
12 Ernest Rossbach 29 DB Senior
13 Latia Vanhoose 28 DB Junior
14 Latia Vanhoose 29 MVC Senior
15 Neda Briski 29 PF Senior
1,000 rows | 5.57s runtime
Refreshed now
14

2. Time Travel

Delta automatically versions your data.

- **Step A: Overwrite with new data**

Example: overwrite with updated status column

df_new = df_transformed.withColumn("status", lit("Updated"))

df_new.write.format("delta").mode("overwrite").save(delta_path)

- **Step B: Read current version (latest)**

df_latest = spark.read.format("delta").load(delta_path)

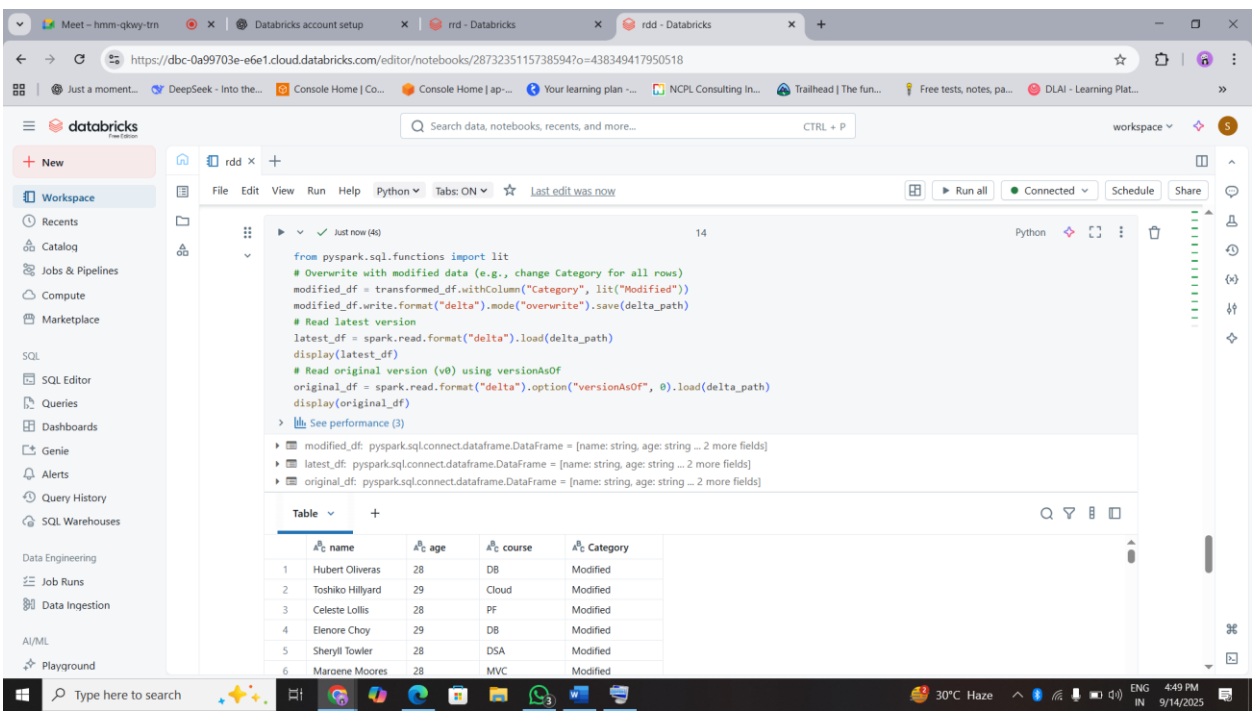
display(df_latest)

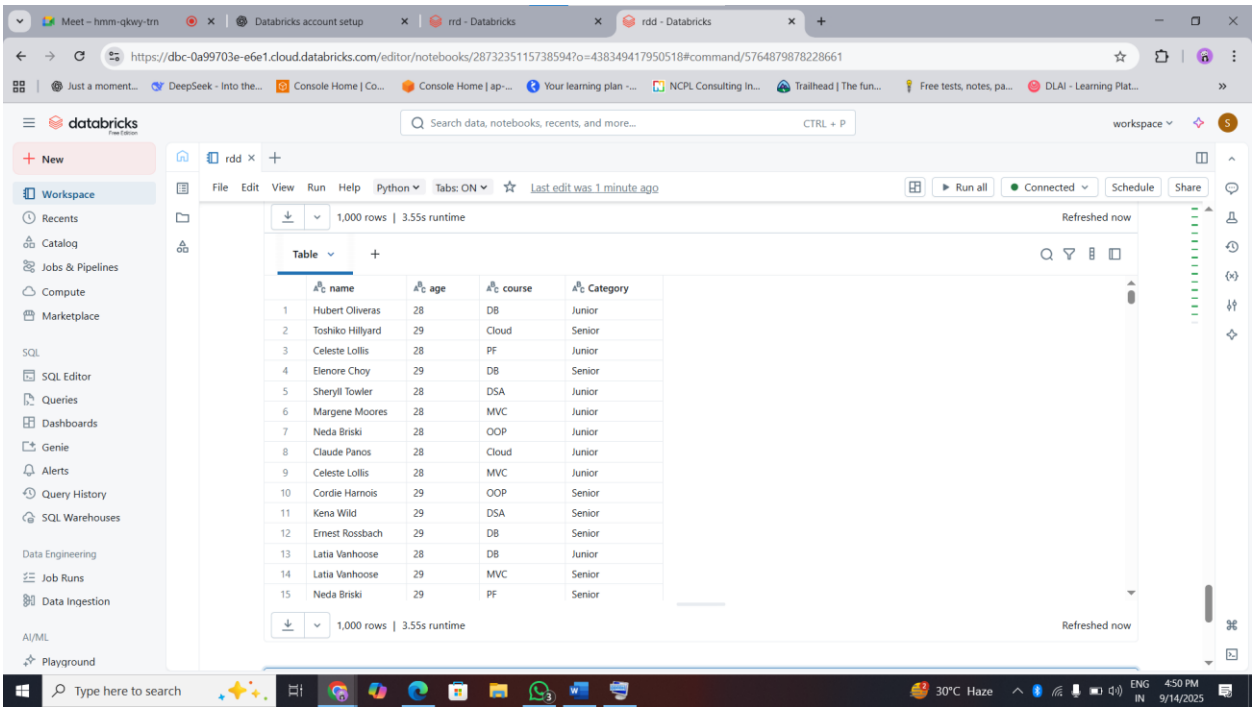
- **Step C: Time Travel to old version (v0)**

df_old = spark.read.format("delta").option("versionAsOf", 0).load(delta_path)

display(df_old)

This shows the original unmodified data (before overwrite).





---

3. Schema Enforcement

Delta enforces schema consistency.

- **Step A: Create a mismatched DataFrame (extra column)**

```
from pyspark.sql.functions import lit

df_extra = df_transformed.withColumn("extra_col", lit("test"))

# Try appending with schema mismatch
df_extra.write.format("delta").mode("append").save(delta_path)
```

This will throw a **schema mismatch error.**

Step B: Fix the schema (make it match)

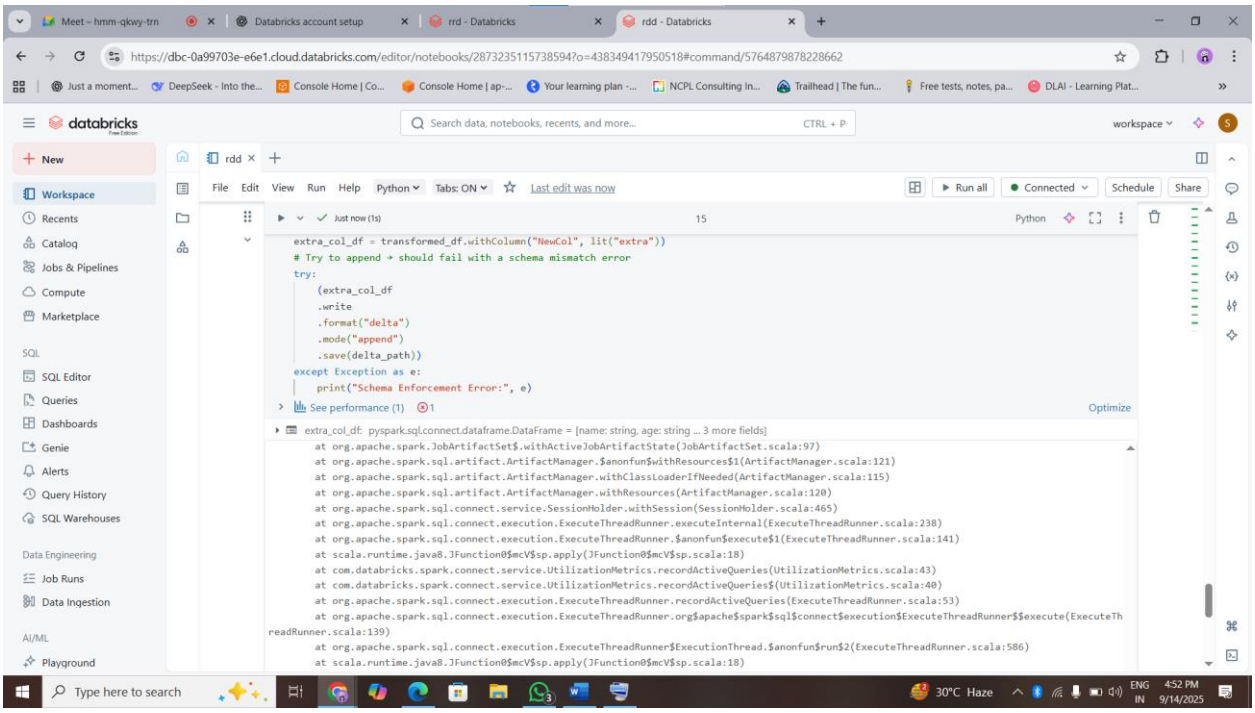
Option 1: Drop the extra column.

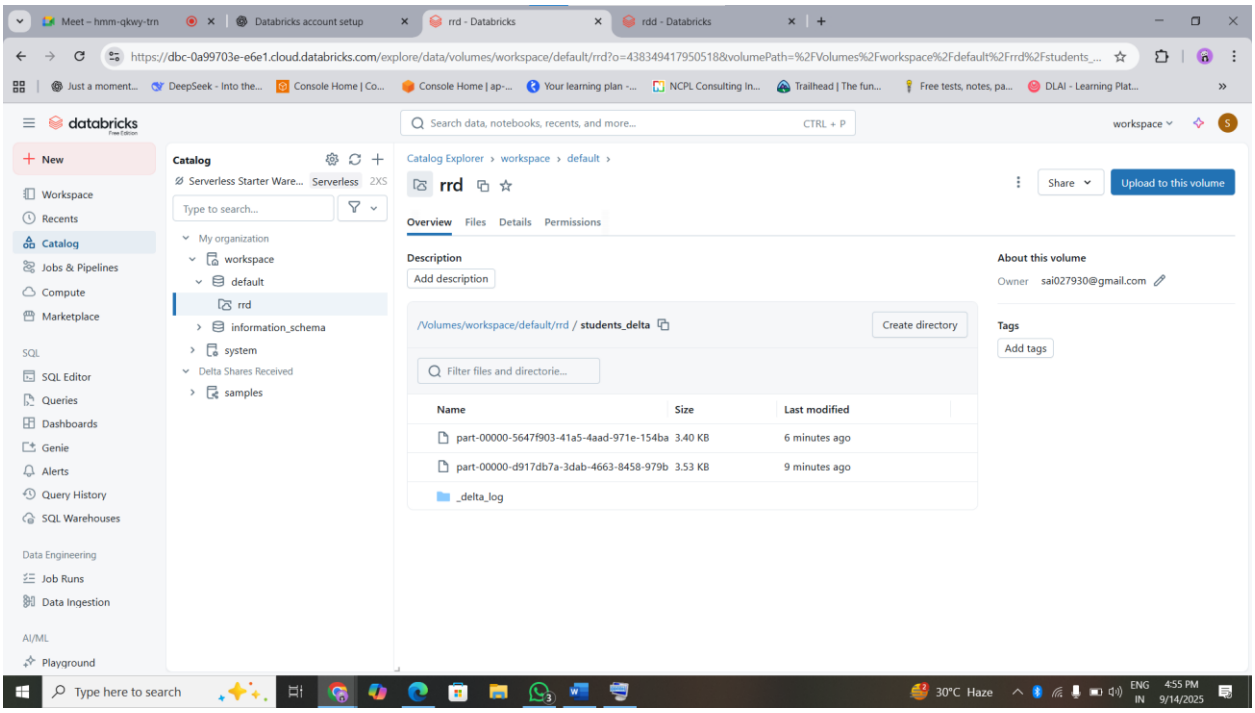
```
df_fixed = df_extra.drop("extra_col")
df_fixed.write.format("delta").mode("append").save(delta_path)
```

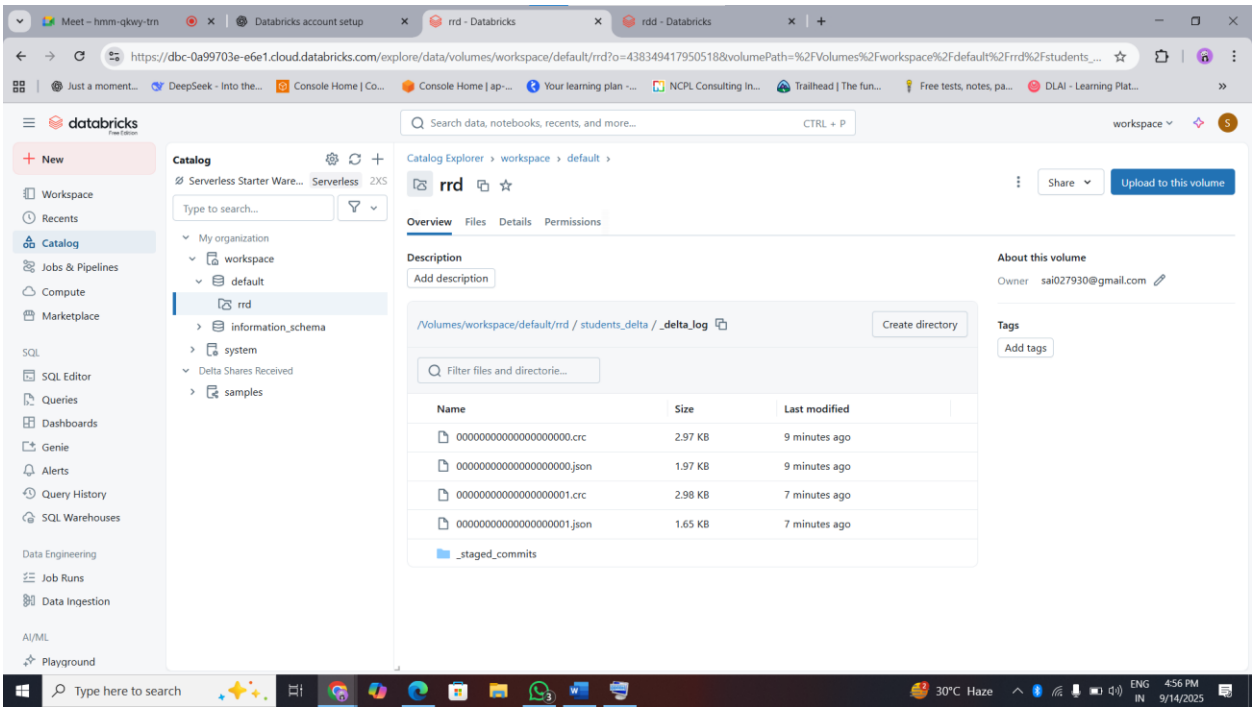
Option 2: Enable **schema evolution** (if new column is intentional).

```
df_extra.write.format("delta").mode("append") \
    .option("mergeSchema", "true") \
```

.save(delta_path)







End Result:

- You created a **Delta table** from a DataFrame.
- Experienced **Time Travel** by querying an older version.
- Observed **Schema Enforcement** (error on mismatched schema).
- Resolved it via schema alignment or evolution.

Spark Structured Streaming – End-to-End Word Count

Theory

Structured Streaming is Spark's scalable and fault-tolerant streaming engine.

Key concepts:

- **Streaming Source** → reads data continuously (here: files in DBFS).
- **Transformations** → apply DataFrame operations like word count.
- **Sink** → write results to console, files, Kafka, Delta, etc.
- **Checkpointing** → Spark saves progress/state so it can recover after failures.
- **Triggers** → define micro-batch intervals (here: every 20s).

Steps

1. Create Input & Checkpoint Volumes

Input folder for streaming text files

" input_path = " /Volumes/workspace/default/stream_input "

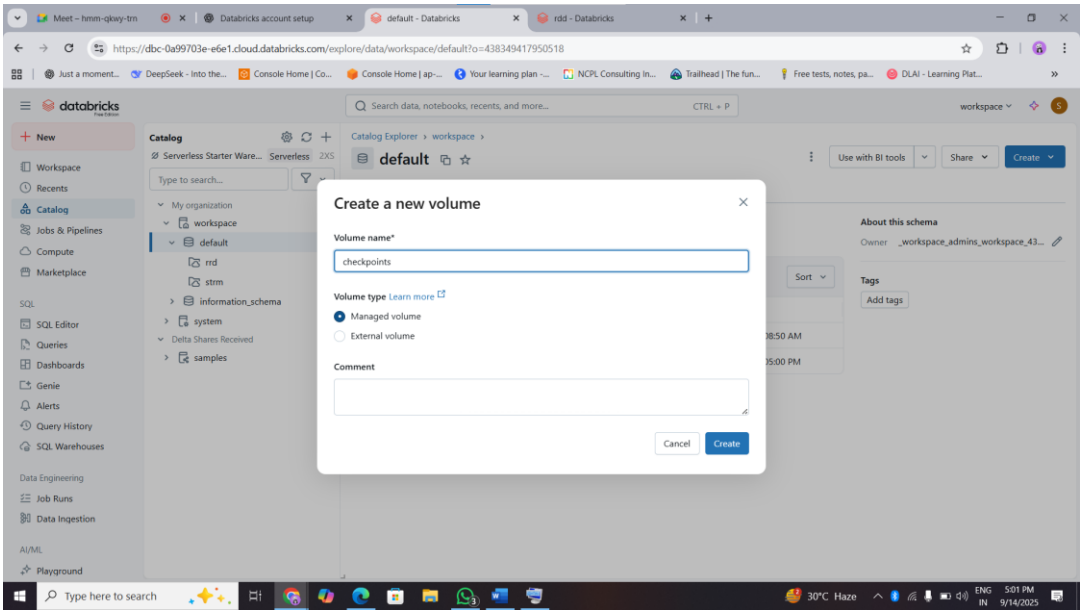
Checkpoint folder for fault tolerance

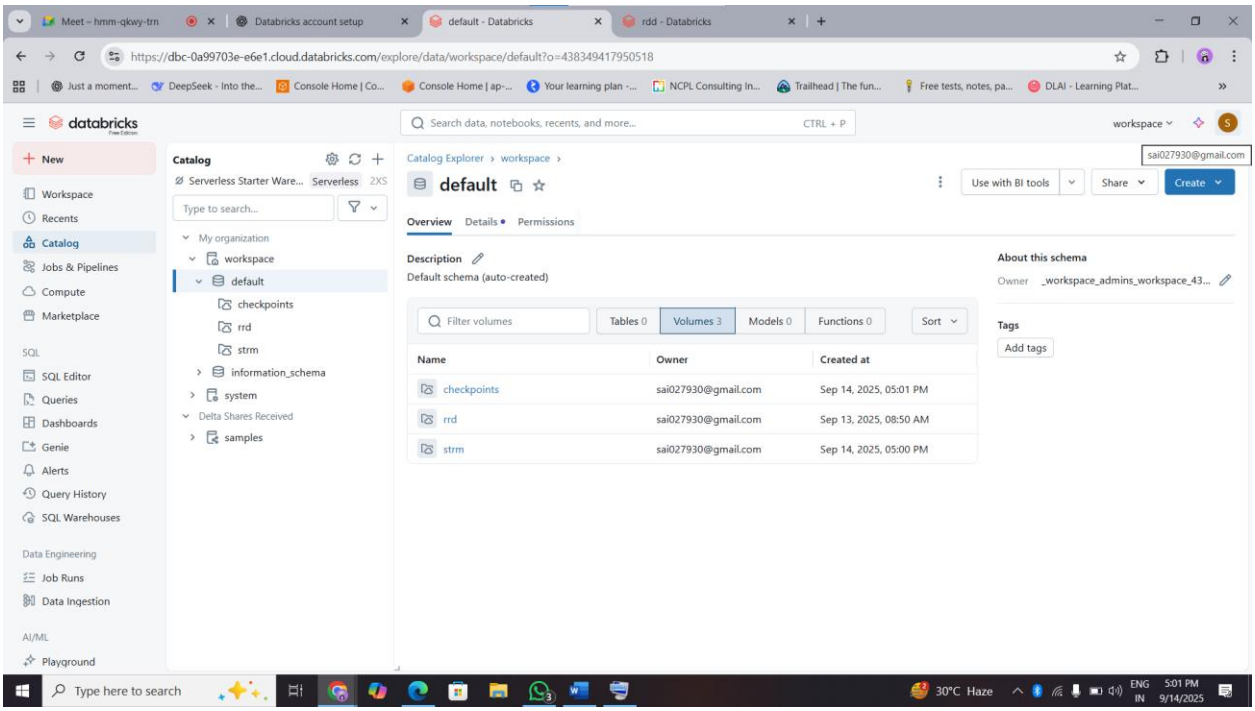
checkpoint_path = " /Volumes/workspace/default/checkpoints "

Create directories in DBFS

dbutils.fs.mkdirs(input_path)

dbutils.fs.mkdirs(checkpoint_path)





---

2. Streaming Word Count Code

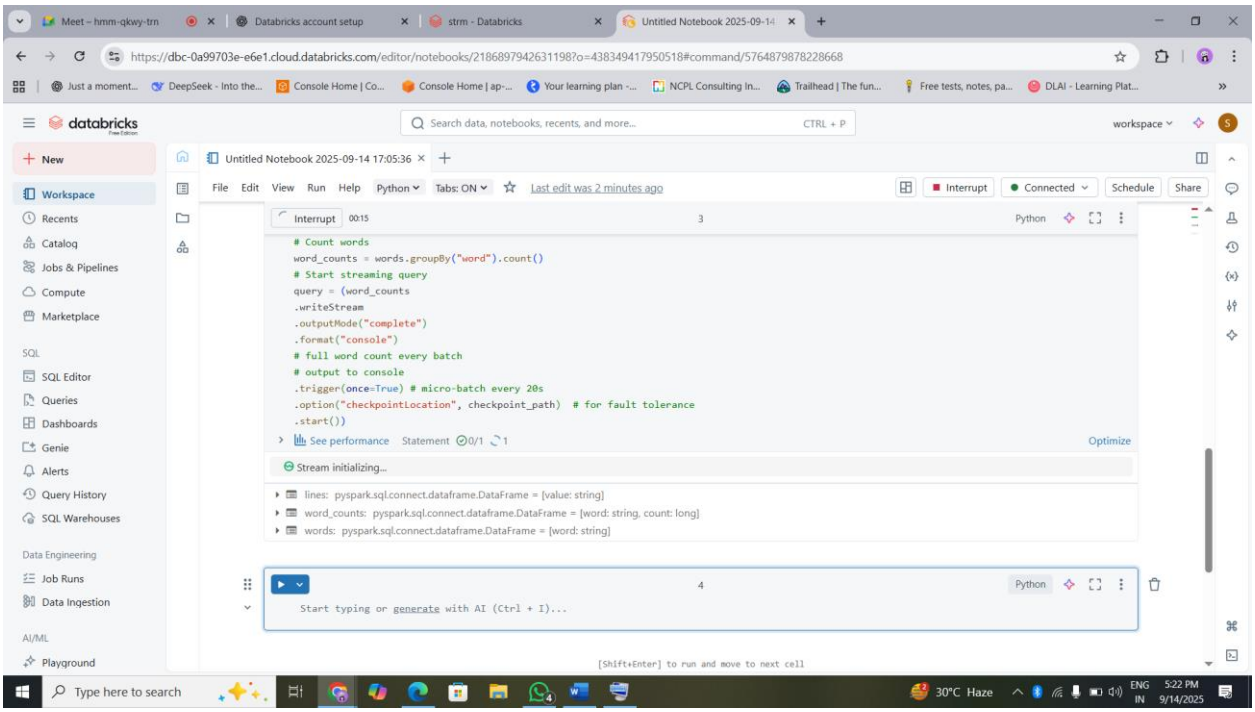
```
from pyspark.sql.functions import explode, split

# Read streaming input (new text files as they arrive)
stream_df = (
    spark.readStream
        .format("text")
        .load(input_path)
)

# Split lines into words
words = stream_df.select(
    explode(split(stream_df.value, " ")).alias("word")
)
# Word count
word_counts = words.groupBy("word").count()
# Write stream to console with checkpointing
query = (
```

```
word_counts.writeStream
    .outputMode("complete")
    .format("console")
    .option("checkpointLocation", checkpoint_path)
    .trigger(processingTime="20 seconds")
    .start()
)
```

Now the query is **running in background**.



---

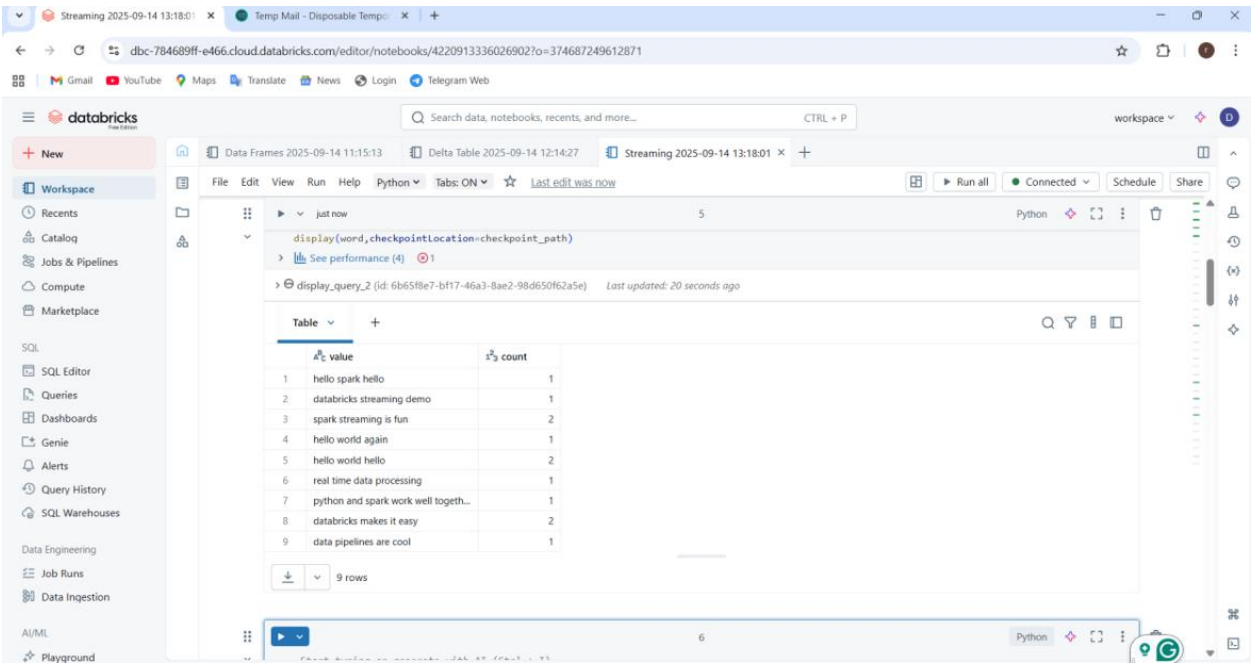
3. Test the Streaming Query

- While the query is active, upload a text file:

dbutils.fs.put("/FileStore/stream_input/file1.txt", "hello world hello spark", True)

- Wait for ~20s.
- Check console output → you should see updated word counts.

---

4. Test Fault Tolerance

1. Stop the query:
2. query.stop()
3. Upload another file:
4. dbutils.fs.put("/FileStore/stream_input/file2.txt", "streaming spark test", True)
5. Restart the query with the same code.
6. Spark will read from the **checkpoint** → it will **not reprocess file1.txt**, only process **file2.txt**.